ForecastLabAI — runtime, CI and production gaps

Problem

Describe how the app is built, run and tested — and what you must NOT claim about production.

Instant solution

Runtime (`docker-compose.yml` , single host)

Service	Port	Note
PostgreSQL 16 + pgvector	host 5433	data + embeddings
FastAPI / Uvicorn	8123	backend
React / Vite	5173	frontend
Ollama	11434	optional, `gpu` profile in `docker-compose.yml`; `docker-compose.gpu.yml` adds the NVIDIA device

Deps: `uv` (Python), `pnpm` (frontend), Alembic (schema migrations). Also `Dockerfile.backend` , `Dockerfile.frontend` , `.devcontainer` , pre-commit.

CI (`.github/workflows/ci.yml`): Ruff check + format, mypy, Pyright, pytest against a pgvector Postgres service, "no pending migrations" check. Other workflows: `cd-release.yml` (release-please), `dependency-check.yml` , `e2e-nightly.yml` , `schema-validation.yml` , `phase-snapshot.yml` .

Do NOT claim: Kubernetes, managed cloud, distributed queue (Celery/broker), centralized tracing, authentication. None are in the repo. Jobs run synchronously behind async-ready contracts; model selection uses in-process background tasks.

"How would you scale it?"

> *Separate API serving from CPU-heavy training with a durable job queue and workers; move artifacts to object storage; add metrics and tracing, authentication and authorization, and data/model drift monitoring. Managed databases and orchestration only once real load justifies the complexity.*

Why it works / gotchas

- `async/await` does not make CPU-bound training non-blocking; it can still block the event loop → worker processes.
- "Application-level MLOps" (registry, aliases, hashes, CI) is the honest framing, not "deployed platform".

Diagram brief

- Type: two side-by-side deployment views.
- Left "Today (single host, Docker Compose)": db:5433 · backend:8123 · frontend:5173 · ollama:11434 (dashed, gpu profile); CI pipeline strip below (ruff → mypy/pyright → pytest+pg → migrations → release-please).
- Right "Scaled (proposal)": API ⇄ job queue → training workers; object storage for artifacts; auth; metrics/tracing; drift monitor. Label it "proposal, not implemented".